

How Search Works on al-nuqta.com

The multilingual hybrid search architecture — embeddings, the Voyage AI system, lexical root matching, and the fallback ladder

MEANING	Voyage AI multilingual embeddings — Arabic and English queries matched to verses by what they <i>mean</i>
ROOTS	The classical lexical arm — queries resolved to tri-consonantal roots and matched through Qur'anic morphology
RESILIENCE	A three-tier fallback ladder — search never breaks, it degrades gracefully and says so

Prepared for the site operator · July 20, 2026

Covers: Phase A (fan-out search) + Phase B (multilingual hybrid retrieval)

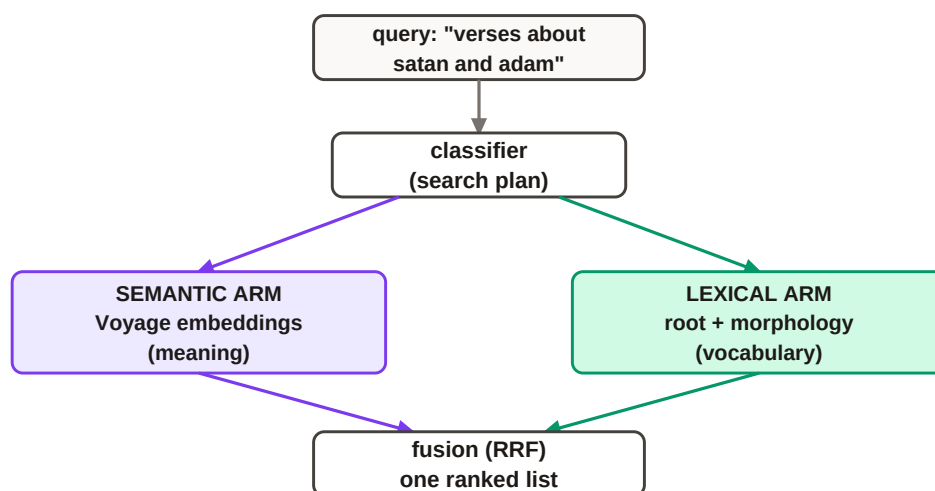
Contents

- 1 The Big Picture — the journey of a query
- 2 Embeddings from First Principles
- 3 The Voyage AI Embedding System
- 4 The Lexical Arm — Roots and Morphology
- 5 Fusing the Two Arms — Reciprocal Rank Fusion
- 6 The Fallback Ladder
- 7 Proving It Works — the Evaluation Harness
- 8 Operations — Keys, Costs, and the Index Lifecycle
- 9 Glossary

1 · The Big Picture

When someone types into the search bar — whether it is **2:255**, **mercy**, **khalaq**, or آيات عن الصبر (“verses about patience”) — the site no longer picks a single kind of search. A small classifier reads the input and produces a **search plan**: which engines should *fire*, and which should *lead* the results. Nothing is forbidden; the plan is about emphasis, not exclusion.

Two engines matter for this document. The **semantic arm** matches by *meaning*, using numerical representations of text called embeddings. The **lexical arm** matches by *vocabulary*, resolving query words to Arabic tri-consonantal roots and finding every verse built on those roots. Each is strong where the other is weak, so their results are fused.



results page: verses + “why it matched” chips

Figure 1 — every conceptual query runs both arms; their rankings are merged.

This shipped in two phases. **Phase A** rebuilt the front half: the classifier, a shareable **/search** results page, recent searches, and fixes to routing. **Phase B** — the subject of most of this document — rebuilt the back half: a multilingual embedding index (so Arabic queries work), the hybrid fusion, and a fallback ladder that keeps search alive when anything upstream fails.

Why two arms? A meaning engine knows that “the frailty of the spider's house” should find 29:41 even though the verse never uses the word “frail.” A root engine knows that الصبر (patience), صابرين (the patient ones), and اصبروا (be patient!) are all the same word family — one root, ر ص ب — something meaning-vectors alone can blur. Together they cover each other's blind spots.

2 · Embeddings from First Principles

An **embedding** is a way of turning a piece of text into a list of numbers — a **vector** — such that texts with similar *meanings* get numerically similar vectors. That single idea powers all modern semantic search.

2.1 · Text becomes a point in space

Imagine plotting every verse as a point on a map, where distance encodes meaning: verses about mercy cluster in one neighborhood, verses about the Day of Judgment in another, Joseph's story in a third. An embedding model is a neural network trained on enormous amounts of text to produce exactly such a map — except the “map” has not two dimensions but hundreds (ours uses **512**). Each dimension captures some learned aspect of meaning; no single dimension means anything human-readable by itself, but together they position the text among everything else the model has read.

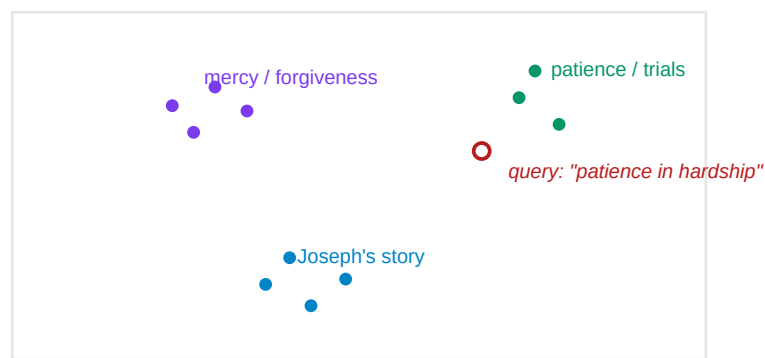


Figure 2 — a 2-D cartoon of a 512-D space: the query lands nearest the verses that mean the same thing.

2.2 · Measuring closeness: cosine similarity

Two vectors are compared with **cosine similarity** — the cosine of the angle between them. It ranges from 1.0 (same direction, same meaning) down to 0 (unrelated) and below. Because every vector is normalized to length 1 first, the cosine reduces to a simple **dot product**, and comparing one query against all 12,472 stored vectors is a single matrix multiplication — a few milliseconds of arithmetic, no database scan.

```
score = query_vector · verse_vector # both L2-normalized, so this IS the cosine
e.g. "mercy" · (1:3 "the All-Merciful, the Ever-Merciful") = 0.57 → shown as "57% match"
```

2.3 · Why the old system could not do Arabic

The first version of semantic search (still on board as a fallback) used a small local model, **all-MiniLM-L6-v2**, which was trained almost entirely on **English**. We embedded each verse's English *translation*. English queries worked well — but an Arabic query got mapped to a essentially random point, because the model had never meaningfully learned Arabic. Our evaluation measured exactly this: on 22 Arabic test queries the old system scored **0.000** — not one relevant verse in any top-10.

The fix is not clever routing — it is a better map. A *multilingual* embedding model places Arabic text and English text in the **same** space: الصبر and “patience” land near each other. That is what Voyage provides.

3 · The Voyage AI Embedding System

Voyage AI is a company (now part of MongoDB) that specializes in embedding models, and it is Anthropic's recommended embeddings provider. Instead of running a neural network on our own server, we call Voyage's hosted API: we send text over HTTPS, it returns the vector. The heavy machine learning runs on their GPUs; our server only stores vectors and does fast arithmetic.

3.1 · Why a hosted model (and which one)

	Old: all-MiniLM-L6-v2	New: voyage-4-lite
Where it runs	on our server (CPU)	Voyage's API (hosted)
Languages	English only	multilingual (incl. Arabic)
Dimensions	384	512 (supports 256–2048)
What we embed	English translation only	Arabic text AND English translation
Arabic queries	noise (eval: 0.000)	native support
Server memory	~90 MB model + torch	none (API call)

voyage-4-lite is the cost/latency-optimized member of Voyage's current family (its larger siblings are voyage-4 and voyage-4-large). It is a *Matryoshka* model: trained so its vectors can be truncated to smaller sizes (256, 512, 1024, 2048) while staying usable. We chose **512 dimensions** — half the storage and double the comparison speed of 1024, with nearly identical retrieval quality at our scale (6,236 verses is tiny by vector-search standards; brute-force comparison is already instant).

3.2 · Two documents per verse — the multi-vector design

Every verse is stored **twice** in the index, as two “documents” embedded into the same space:

- **ar** — the Uthmani Arabic text itself (Bismillah-stripped for display consistency)
- **en** — the best available English translation, plus the verse's curated theme tags

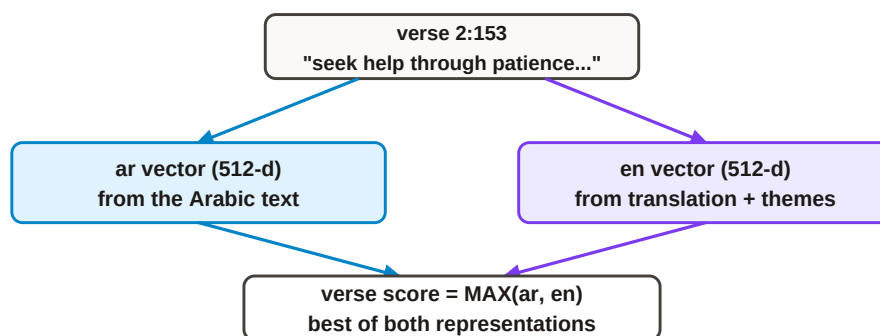


Figure 3 — one verse, two vectors. A query is compared against both; the verse takes its better score.

At query time the incoming text — Arabic or English — is embedded once and compared against *all* vectors. Each verse's score is the **maximum** over its two documents. An Arabic query usually resonates most with ar vectors; an English query with en vectors; but cross-language matches happen freely because it is all one space — that is precisely what “multilingual” buys.

3.3 · Queries and documents are embedded differently

Voyage's API takes an **input_type** parameter: **document** when indexing verses, **query** when embedding a search. Under the hood the model prepends a different instruction for each, nudging queries and documents toward each other when they are about the same thing. It is a small asymmetry that measurably improves retrieval.

```
POST https://api.voyageai.com/v1/embeddings
{ "model": "voyage-4-lite", "input": ["...verse text..."],
  "input_type": "document", "output_dimension": 512 }
```

3.4 · The index at rest

The 12,472 vectors (6,236 verses × 2 documents) live in an ordinary SQLite table, **verse_embeddings_v2**, each vector stored as a 2 KB binary blob (512 numbers × 4 bytes) alongside the exact text that produced it and a hash of that text. On startup the server loads them into one in-memory matrix (~25 MB). The hash makes indexing *resumable*: re-running the indexer re-embeds only verses whose text changed.

4 · The Lexical Arm — Roots and Morphology

The second arm needs no API and no neural network. It leans on the most distinctive feature of Arabic — and the founding idea of this site: almost every word derives from a **tri-consonantal root**, and the Qur'an's morphology has been fully annotated for a century. The site already maintained a root index for its word-tracing features; search reuses it.

4.1 · From query words to roots

Each query word is resolved to candidate roots. For Arabic words the trick is the **consonantal skeleton**: strip the diacritics, normalize letter variants, remove the definite article, and check which root's letters appear inside the word:

query word	normalized	root found	meaning
الصبر	صبر	ص ب ر (Sbr)	endurance, patience
الرحمة	رحمه	ر ح م (rHm)	mercy (womb etymology)
والمغفرة	مغفره	غ ف ر (gfr)	forgiveness, covering

English words take a different path: they are matched against the roots' English meaning glosses (“mercy” → rHm) and against Buckwalter transliterations (typing “Sbr” directly names a root). Filler words — “the,” “verses,” آيات, عن — are dropped first.

4.2 · From roots to verses, weighted by rarity

The morphology index knows every verse each root appears in. Matches are scored with **IDF** (inverse document frequency): a rare root is a strong signal, a ubiquitous one is weak. A verse matching several of the query's roots accumulates their weights and rises. This is classical search-engine mathematics — no machine learning — which is exactly why it is a dependable fallback: it cannot time out, cannot be rate-limited, and costs nothing.

Why keep a lexical arm at all, if embeddings are so capable? Three reasons. (1) Precision on rare vocabulary: for a query naming a specific root, exact root match beats semantic blur. (2) Explainability: the emerald “root” chip on a result is a statement of fact, not a similarity score. (3) Independence: it is the arm that still works when the network does not.

5 · Fusing the Two Arms — Reciprocal Rank Fusion

The two arms produce two ranked lists with incomparable scores — a cosine like 0.57 and an IDF sum like 3.8 live on different scales. **Reciprocal Rank Fusion (RRF)** merges them using only each verse's *rank* in each list, which sidesteps the scale problem entirely:

```
fused(verse) = 1.0 × 1/(60 + rank_dense) + 0.7 × 1/(60 + rank_lexical)
```

The constant 60 is the standard damping factor from the RRF literature: it keeps a #1 rank from dominating and makes agreement between lists the main force. The 0.7 weight tilts slightly toward the semantic arm. A verse that appears in **both** lists gets both contributions — so verses that are semantically close *and* share a root with the query float to the top. A worked example:

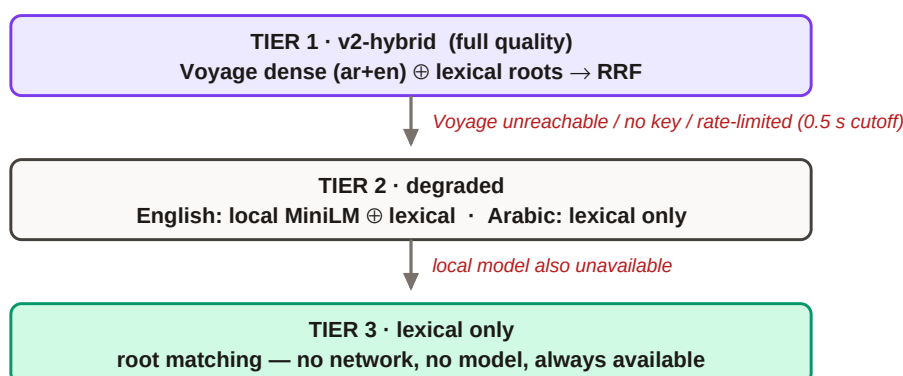
verse	dense rank	lexical rank	RRF score	final
6:54 (mercy written upon Himself)	#2	#1	$1/62 + 0.7/61 = .0276$	#1
1:3 (the All-Merciful)	#1	—	$1/61 = .0164$	#2
16:119 (forgiving after evil)	#4	#3	$1/64 + 0.7/63 = .0267$	#3 → #2*

**Order among close scores shifts with the exact ranks; the table illustrates the mechanics.*

The interface shows the **dense cosine** as the “% match” badge (RRF scores are tiny fractions that would confuse), and renders **matched_because** chips: a sky “Arabic” chip when the verse’s Arabic vector was the best match, an emerald “root” chip when the lexical arm agreed. The goal is that a result never feels like an oracle — it always says *why* it is there.

6 · The Fallback Ladder

The dense arm depends on an external API — which means timeouts, rate limits, and outages are a matter of *when*, not *if*. The design rule, learned from a real production incident in July 2026, is: **search may degrade, but it must never break, and it must never hang.**



every response carries "degraded": true/false and "engine": "..."

Figure 4 — the ladder. Each step down trades quality for certainty.

6.1 · How the guardrails work

- **Hard timeouts:** the query-embedding call gives Voyage 3 seconds to accept the connection and 6 to answer; any failure returns *None* and the ladder steps down. No exception ever escapes to the user.
- **Bounded concurrency:** at most 4 embedding calls in flight; a fifth waits at most 0.5 s, then degrades.

A slow upstream can never pile up threads (the failure mode behind the July 2 outage).

- **Query cache:** the last 512 embedded queries are kept in memory, so repeated and recent searches skip the API entirely.
- **Language-aware degradation:** the tier-2 local model is English-only, so an Arabic query skips it — falling to lexical rather than through it (which would return noise dressed as results).
- **Abuse guard:** 30 searches per minute per IP; beyond that the API answers 429 rather than letting one client drain the Voyage budget.

Honesty as a feature: the degraded flag is not cosmetic. During the days before the Voyage index was built, production ran the ladder's lower tiers — and the evaluation showed even *that* beat the old system. The upgrade to full quality was a pure data sync, with zero code changes.

7 · Proving It Works — the Evaluation Harness

“The search feels better” is not evidence. Before any cutover, a standalone evaluation harness scores each engine against a hand-labeled **gold set**: 50 conceptual queries (28 English, 22 Arabic), each with the verses a good engine should find — e.g. “the army of the elephant destroyed by birds” → Surah 105; أصحاب الكهف → 18:9–25.

7.1 · The metrics

- **recall@k** — of the query's known-relevant verses, what fraction appear in the top k results?
- **hit@k** — did at least one relevant verse make the top k? (the “did the user find *anything*” metric)
- **MRR** — mean reciprocal rank of the first relevant verse: 1.0 if it is #1, 0.5 if #2, 0.33 if #3 ... (the “how far do I scroll” metric)

7.2 · The numbers that justified the cutover

	MRR	hit@10 (EN)	hit@10 (AR)	verdict
v1 — old MiniLM semantic	0.257	0.786	0.000	Arabic totally blind
v2 degraded — lexical arm only (before the Voyage index)	0.365	0.750	0.364	already beats v1 overall; Arabic lifts off zero
v2 full — Voyage dense + lexical (live)	0.488	0.857	0.727	Arabic genuinely works; ~2x the lexical tier

The middle row is the key decision datum: the hybrid architecture *without* its dense arm — running purely on the lexical ladder tier — already outperformed the old engine overall (MRR 0.257 → 0.365) while taking Arabic from literally nothing to a working search. That made shipping safe immediately, with the Voyage index as a follow-on quality upgrade rather than a launch dependency. The bottom row is what is **live now**: adding the Voyage dense arm nearly **doubled** overall MRR versus the old engine (0.257 → 0.488) and took Arabic hit@10 to **0.727** — from a search that returned nothing to one where seven in ten conceptual Arabic queries surface a relevant verse near the top.

A gold set is a lab instrument, not a report card. Its absolute numbers depend on how strict the labels are; its value is *comparing* engines on identical terms and catching regressions before users do. The same harness runs before every future search change.

8 · Operations — Keys, Costs, and the Index Lifecycle

8.1 · The API key

The Voyage key lives in **Admin** → **Settings** → **AI providers** → **Voyage AI (search)** (stored in the site database, cached for 60 s, never in git). The indexer on the development machine reads it from a git-ignored **.env** file. If the key is removed or expires, nothing breaks — the ladder simply holds at tier 2/3 and responses say *degraded: true*.

8.2 · Cost and rate limits

Embedding is priced per token. Indexing the entire Qur'an twice over (ar + en) is only about **500K tokens** — a fraction of a percent of Voyage's free-tier token grant — and a live query costs a few dozen tokens more. The practical constraint is **rate**, not money: an account with no payment method on file is capped at

3 requests and 10K tokens per minute, which slows a full index build to about two hours and throttles how many live queries per minute get tier-1 treatment (the rest gracefully degrade). Adding a payment method on Voyage's billing page lifts the caps to standard levels (the build drops to ~2 minutes) while the token grant keeps actual cost at zero.

8.3 · The index lifecycle

1. **Build** (dev machine): `python build_embeddings_v2.py` — embeds all verse documents, commits per batch, resumable, self-paces under rate limits.
2. **Evaluate**: `python eval_retrieval.py --engine v1 --engine2 v2` — the cutover gate.
3. **Ship**: `./sync_tables_to_prod.sh verse_embeddings_v2` — copies ~25 MB of vectors to production.
4. **Restart** the production container — it loads the matrix at startup and tier 1 lights up. No code deploy is involved; the index is pure data.
5. **Re-index later** only when verse translations change — the text-hash makes that incremental.

9 · Glossary

embedding	A learned mapping from text to a vector of numbers such that similar meanings yield nearby vectors.
vector / dimension	The list of numbers itself; each of its 512 positions is one 'dimension' of the meaning space.
cosine similarity	Closeness measure between two vectors (the cosine of their angle); 1.0 = same direction.
dot product	The multiply-and-sum operation that computes cosine similarity for normalized vectors.
multilingual model	An embedding model trained across languages so that translations land near each other.
Matryoshka embeddings	Vectors trained to remain useful when truncated to fewer dimensions (named after nesting dolls).
input_type (query/document)	Voyage's switch that embeds searches and corpus text slightly differently, improving retrieval.
dense / lexical	'Dense' = embedding-based matching; 'lexical' = word/root-based matching.
tri-consonantal root	The (usually) three-consonant core from which Arabic word families derive, e.g. ص ب ر → patience words.
IDF	Inverse document frequency — rare terms weigh more than common ones.
RRF	Reciprocal Rank Fusion — merging ranked lists using ranks instead of raw scores; damping constant 60.
recall@k / hit@k / MRR	Retrieval quality metrics: coverage in top-k, any-hit in top-k, and rank of the first hit.
gold set	Hand-labeled query→relevant-verses pairs used to score engines objectively.
degraded mode	Serving from a lower ladder tier and saying so in the response, instead of failing.
rate limit (RPM/TPM)	Provider caps on requests and tokens per minute; free-tier Voyage is 3 RPM / 10K TPM.
Buckwalter	An ASCII transliteration scheme for Arabic (ص ب ر = 'Sbr'), used as the site's canonical root key.

Generated July 20, 2026 · reflects Phase A (fan-out search) and Phase B (multilingual hybrid retrieval), both live on al-nuqta.com · the Voyage dense index (12,472 vectors) is built and serving; §7.2 shows the measured before/after.